ROS2 Manipulation Basics

Unit 5: Object Detection

- Summary -

Estimated time for completion: **2h**

This unit will show you how to perform motion planning with Python. By completing this unit, you will be able to create a Python program that performs motion planning on your robot.

- End of Summary -

5.1 What is Perception?

In robotics, we understand **perception** as how robots can perceive the world that surrounds them based on input sensory data. This perception of the environment is very helpful for robots to perform any required task.

When working with manipulator robots, one of the main goals of reach is picking an object up from one position and placing it in another, commonly known as pick & place. Although this may sound like a very simple task, it is not. In fact, it's a very complex process because lots of variables need to be taken into account when picking up the object. For instance, how does the robot know where the object to pick is placed?

Well, the goal of this unit is to teach you how to answer this question.

5.1.1 The simple_grasping package

In this unit, you will be using the **simple_grasping** ROS2 package. Mike Ferguson developed this package, so a big shoutout to his amazing work. For this course, we have done some modifications to the original code to better adapt it to the material, but you can check the original code here: https://github.com/mikeferguson/simple_grasping/tree/ros2

This package is very useful for manipulation since it will handle the detection of the object/s to pick, and it can also generate the grasp poses for many types of objects. And it's pretty simple, which helps in the process of learning. This package gets the object's position to be picked from a 3D camera sensor (PointCloud) and generates the necessary grasping sequences to pick the object up. However, you will focus on the Object Detection part of this unit, and leave aside all the grasping parts.
So, the first thing you are going to do is get the code of the **simple_grasping** package. To do so, you can execute the following commands:

Execute in Shell

In [ ]:

```
cd ~/ros2_ws/src
git clone https://bitbucket.org/theconstructcore/simple_grasping_ros2.git
cd ..
colcon build
source install/setup.bash
```

Now, inside the **simple_grasping** package, you will see a **src** folder. Here is where all the main programs for the package are located. In this chapter, though, you are going to focus on two of them:

- **basic_grasping_perception.cpp**: This script is written in C++. It provides an action server named `/find_objects`, which can be called to trigger the object detection pipeline.
- **object_support_segmentation.cpp**: This script is written in C++. Its main function is to process the data received from the PointCloud 3D camera to detect any pickable objects in the scene.

So, in the following sections of this unit, you will have a deeper look into these two scripts and how they work.

5.1.2 Depth Camera Sensor

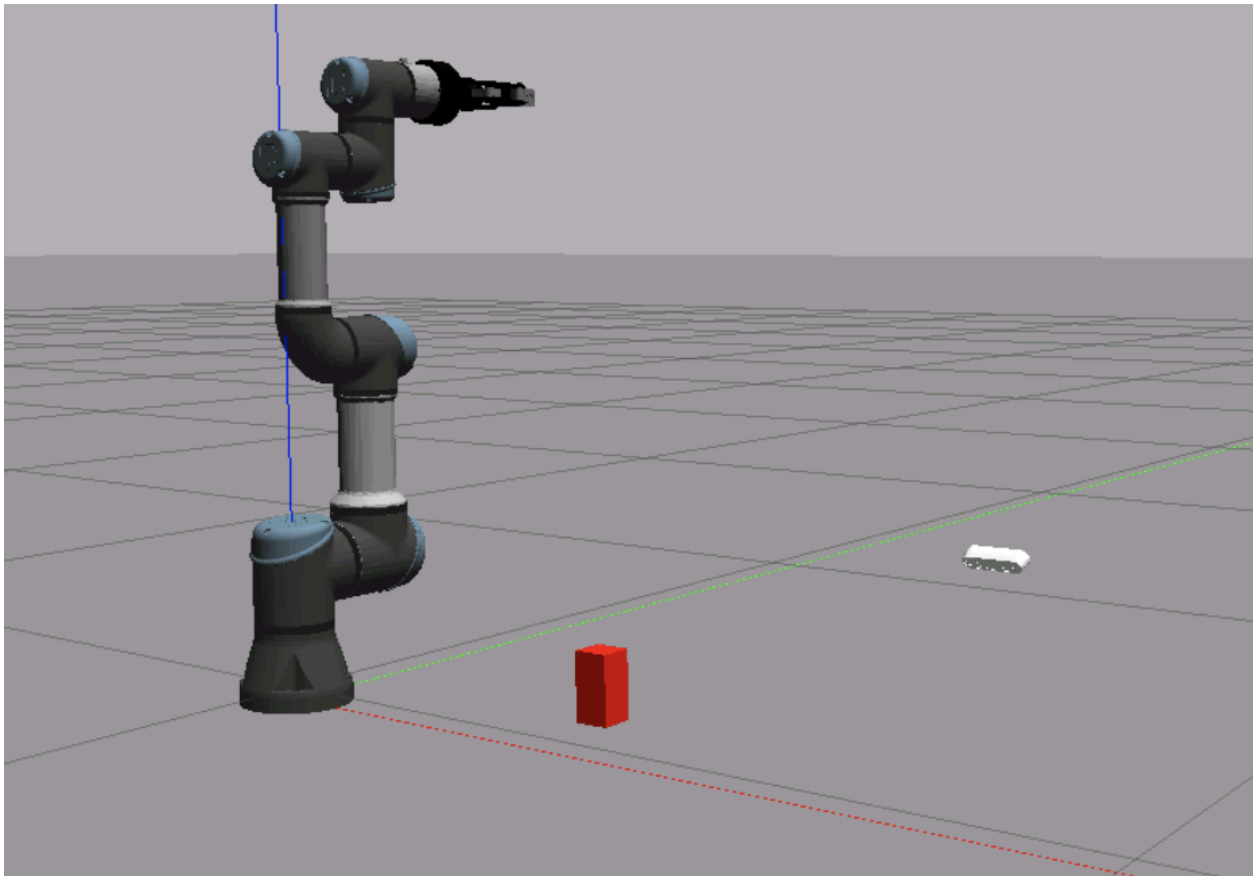
First of all, execute the following command to spawn a pickable object into the simulation.

Execute in Shell

In [ ]:

```
ros2 run gazebo_ros spawn_entity.py -file ~/ros2_ws/src/grasp_box.urdf -x 0.5 -y 0.13 -z 0.1 -entity grasp_box
```

In the end, you should have a grasping cube right in front of the UR3e arm, like in the image below:



Great! So, now you are ready to start.
Let's start by analyzing the data published by the Depth Camera. As you may imagine, this data is being published in ROS1. Therefore, you can easily check the different topics published by the camera by running the following commands:

Execute in Shell

In [ ]:

```
ros2 topic list
```

Expected Output

In [ ]:

```
...

/wrist_rgbd_depth_sensor/camera_info
/wrist_rgbd_depth_sensor/depth/camera_info
/wrist_rgbd_depth_sensor/depth/image_raw
/wrist_rgbd_depth_sensor/depth/image_raw/compressed
/wrist_rgbd_depth_sensor/depth/image_raw/compressedDepth
/wrist_rgbd_depth_sensor/depth/image_raw/theora
/wrist_rgbd_depth_sensor/image_raw
/wrist_rgbd_depth_sensor/image_raw/compressed
/wrist_rgbd_depth_sensor/image_raw/compressedDepth
/wrist_rgbd_depth_sensor/image_raw/theora
```

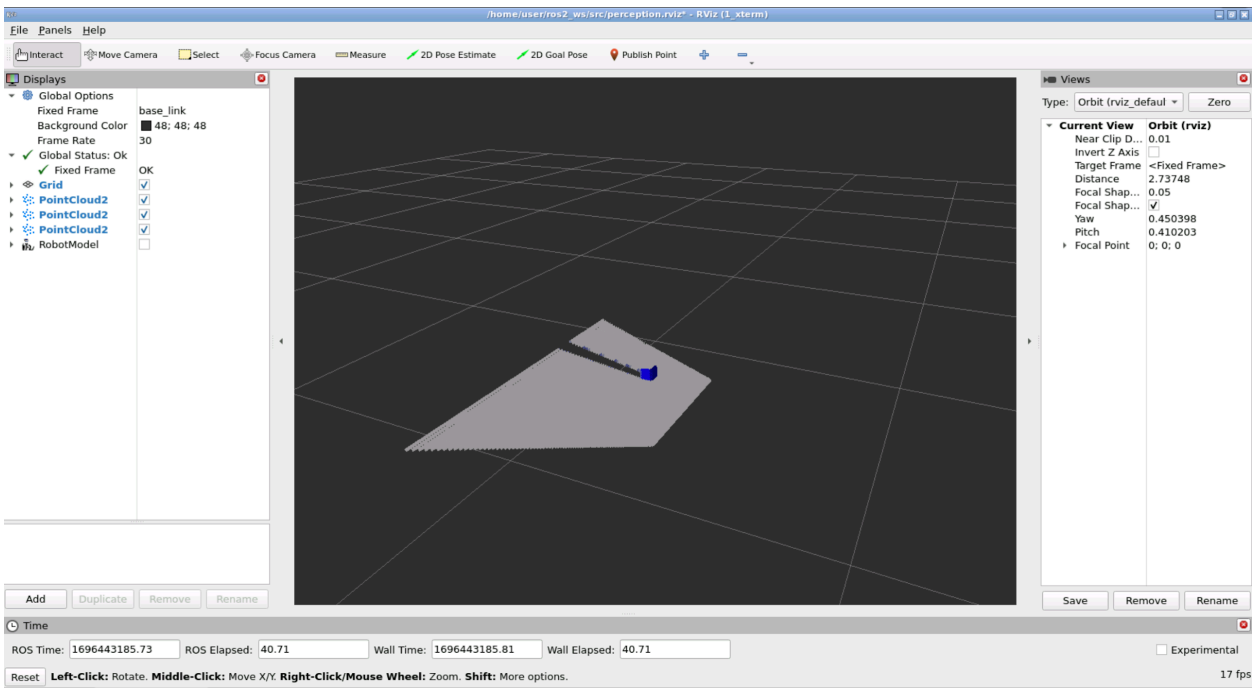
/wrist_rgbd_depth_sensor/points

As you can see, different topics are being published by the camera. In this case, the topic you are most interested in is the one named **/wrist_rgbd_depth_sensor/points**. Why? Well, this topic is the one that contains the Depth Data (PointCloud). This PointCloud will be used by your Perception programs (**simple_grasping**) to detect the objects in the scene.
Finally, verify that the topics are being properly bridged by visualizing the PointCloud data in RVIZ2.
Execute in Shell

In [ ]:

```
rviz2
```

Add a `PointCloud2` display to RViz2 in order to visualize the generated PointCloud data. You should get something similar to this:



5.2 Running the Object Detection action

The command to start the Object Detection Action Server is the following:
Execute in Shell

In [ ]:

```
ros2 run simple_grasping basic_grasping_perception_node --ros-args -p debug_topics:=true
```

You are starting a ROS2 node named `basic_grasping_perception_node`, the one contained in the **basic_grasping_perception.cpp** script.

- Exercise 5.1: Verify the action server has been properly started -

- When the program is running, a new action server is created. You can check it by using the ***ros2 action list*** command.

Execute in Shell

In [ ]:

```
ros2 action list
```

Here you can see an Action Server named **find_objects**:

Expected Output

In [ ]:

```
/find_objects
```

By calling this Action Server, you ask the robot to find any object close to him that is graspable. You will do this in a moment.

Also, if you run a `ros2 topic list` command, you will see two extra topics:

In [ ]:

```
/object_cloud
/support_cloud
```

These two topics allow you to visualize what the robot detects using RVIZ2. The **/object_cloud** shows you what the objects detected, and the **/support_cloud** shows you the surface where these objects are placed.

- End Exercise 5.1 -

- Exercise 5.2: Call the action server to ACTIVATE the object detection -

Next, CALL this Action Server to see if it detects an object. For that, you'll need to run the following command:

Execute in Shell

In [ ]:

```
ros2 action send_goal /find_objects grasping_msgs/action/FindGraspableObjects "{plan_grasps: false}"
```

The parameter **plan_grasps** indicates whether you want to calculate the possible grasps or not.

By sending a goal to the **/find_objects** Action Server, you trigger the system to look for objects around the scene. After some seconds you will get a result message similar to the following:

Expected Output

In [ ]:

```
Waiting for an action server to become available...
Sending goal:
    plan_grasps: false

Goal accepted with ID: 8700ae9645314b0d98018b446a2818f8

Result:
   objects:
- object:
   header:
     stamp:
```

```
      sec: 1634920618
      nanosec: 76980389
    frame_id: base_link
  name: ''
  support_surface: surface0
  properties: []
  point_cluster:
    header:
      stamp:
        sec: 0
        nanosec: 0
      frame_id: ''
    height: 1
    width: 1760
    fields:
    - name: x
      offset: 0
      datatype: 7
      count: 1
    - name: y
      offset: 4
      datatype: 7
      count: 1
    - name: z
      offset: 8
      datatype: 7
      count: 1
    - name: rgb
      offset: 16
      datatype: 7
      count: 1
    is_bigendian: false
    point_step: 32
    row_step: 56320
    data:
    - 103
    - 28
    - 208

...

    - 128
    - 63
    is_dense: true
  primitives:
  - type: 1
```

```
    dimensions:
    - 9.568283081054688
    - 4.955076992511749
    - 0.007858745753765106
  primitive_poses:
  - position:
      x: 0.48848390579223633
      y: -2.0752249658107758
      z: -0.09606975689530373
    orientation:
      x: 0.0
      y: 0.0
      z: 0.0
      w: 1.0
  meshes: []
  mesh_poses: []
  surface:
    coef:
    - -2.9003527743043378e-05
    - 2.6923982659354806e-05
    - 1.0
    - 0.10009085386991501

Goal finished with status: SUCCEEDED
```

As you can see, the result message returned by the Action Server is HUGE. So let's try to understand it a little bit better. You know that the Action Server uses the interface `grasping_msgs/action/FindGraspableObjects`, so let's get some more data:

Execute in Shell

In [ ]:

```
ros2 interface show grasping_msgs/action/FindGraspableObjects
```

Expected Output

In [ ]:

```
##############################################################
# This action is called for integrated object detection and
#  grasp planning, such as in base_grasping_perception

# Set to false to disable grasp planning, returning only the objects found
bool plan_grasps
---
# Graspable objects found
GraspableObject[] objects

# Additional, non-graspable objects which may be support surfaces
```

```
Object[] support_surfaces
---
# Publish objects as they are detected and grasp planned
GraspableObject object
```

So you can see that the result message contains two arrays, one for the objects detected, `GraspableObject[] objects`, and another one for the surfaces, `Object[] support_surfaces`.

The result data is not easy to understand as it is. What if you ask RVIZ2 for some help here?

- End Exercise 5.1.2 -

- Exercise 5.1.3: Visualize the detections in RViz2 -

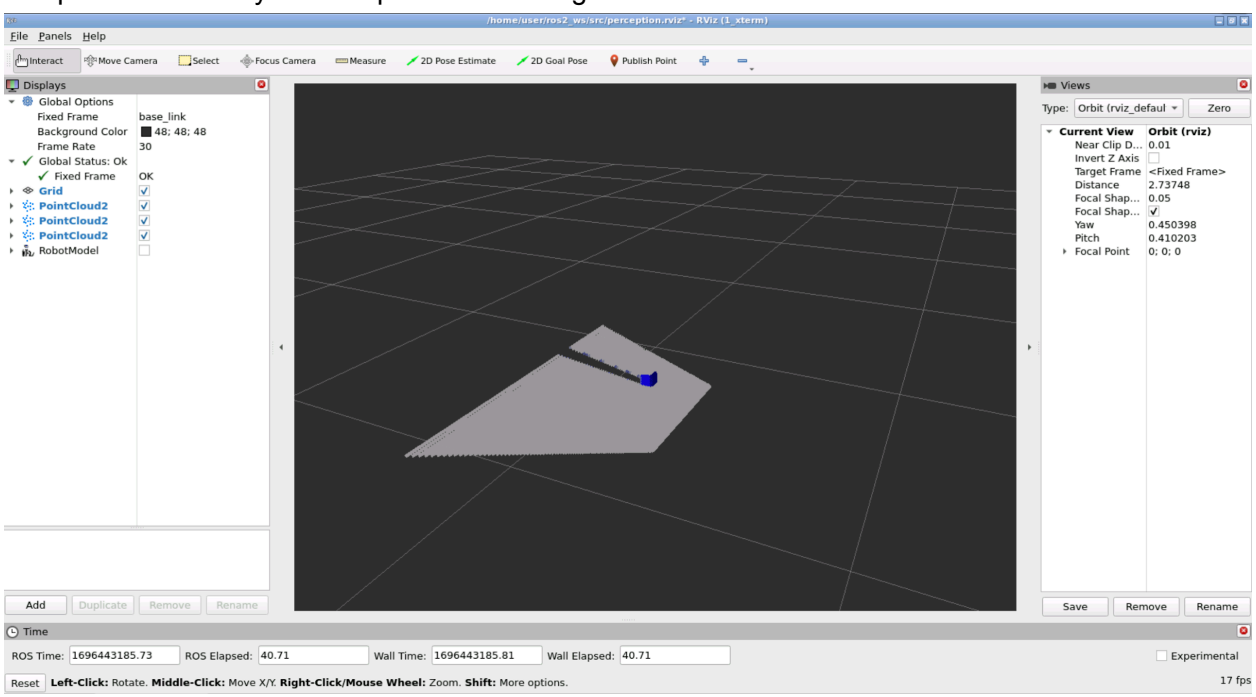
Open RViz2 if you don't have it already running.

Execute in Shell

In [ ]:

```
rviz2
```

Setup RViz2 so that you end up with something like this:

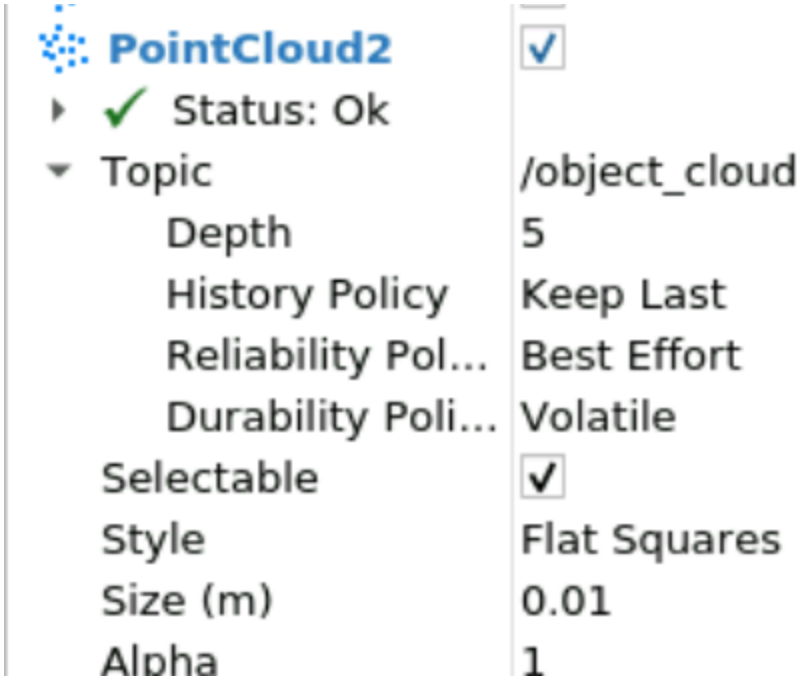


Now, add two additional PointCloud displays to visualize the following two topics:

In [ ]:

```
/object_cloud
/support_cloud
```

Make sure to set the **Reliability Policy** of the Topics to **Best Effort**. Otherwise, you won't visualize the detections.



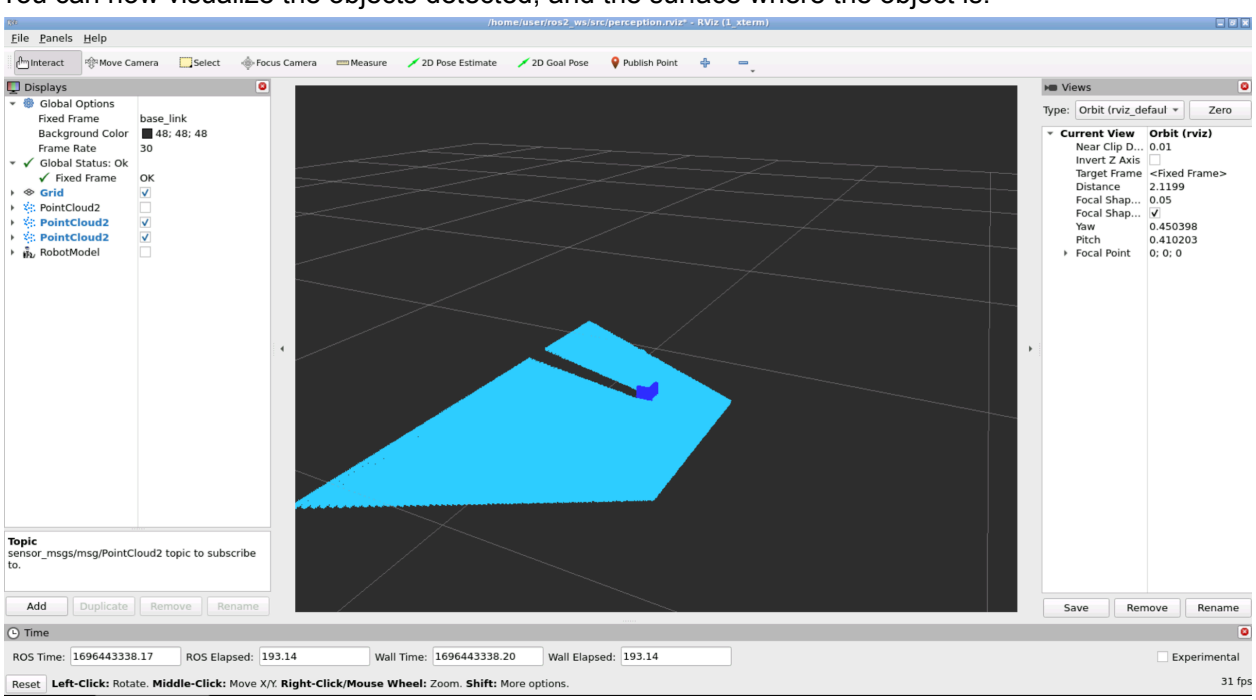
Now you will need to send a goal again to the Action Server:

Execute in Shell

In [ ]:

```
ros2 action send_goal /find_objects grasping_msgs/action/FindGraspableObjects "{plan_grasps: false}"
```

You can now visualize the objects detected, and the surface where the object is.



You should now save this **RVIZ2 configuration file** with the name **perception.rviz**

- End Exercise 5.1.3 -

5.2.1 Reviewing the code of basic_grasping_perception.cpp

Let's look at the most relevant parts of the code of **src/basic_grasping_perception.cpp** in the package **simple_grasping** that you cloned in the previous section.

In [ ]:

```
debug_ = this->declare_parameter<bool>("debug_topics", false);

if (debug_)
{
  rclcpp::QoS qos(1);
  qos.best_effort();
  object_cloud_pub_ =
    this->create_publisher<sensor_msgs::msg::PointCloud2>("object_cloud", qos);
  support_cloud_pub_ =
    this->create_publisher<sensor_msgs::msg::PointCloud2>("support_cloud", qos);
}
```

With the following lines, you can enable/disable the output of a cloud containing the points of the object detections. This is why you set the **debug_topics** parameter to **true** in the launching command.

Then, if the **debug_topics** parameter is set to **true**, two new topics will be advertised containing the PointCloud data of the detected objects and their surface.

In [ ]:

```
world_frame_ = this->declare_parameter<std::string>("frame_id", "base_link");
```

Here, set the **world frame** to **base_link**. Therefore, all the object detections will be referenced to the **base_link** frame.

In [ ]:

```
rclcpp::QoS points_qos(10);
points_qos.best_effort();
cloud_sub_ = this->create_subscription<sensor_msgs::msg::PointCloud2>(
  "/wrist_rgbd_depth_sensor/points",
  points_qos,
  std::bind(&BasicGraspingPerception::cloud_callback, this, _1));
```

Here, subscribe to the PointCloud of the Depth Camera, which is published in the **/wrist_camera/depth/points** topic.

In [ ]:

```
server_ = rclcpp_action::create_server<FindGraspableObjectsAction>(
      this->get_node_base_interface(),
      this->get_node_clock_interface(),
      this->get_node_logging_interface(),
      this->get_node_waitables_interface(),
```

```
    "find_objects",
    std::bind(&BasicGraspingPerception::handle_goal, this, _1, _2),
    std::bind(&BasicGraspingPerception::handle_cancel, this, _1),
    std::bind(&BasicGraspingPerception::handle_accepted, this, _1)
  );
```

Here, advertise the Action Server you used in the previous exercise, named **find_objects**. As you already know, this Action Server **triggers the whole process of detecting the objects**.

In [ ]:

```
void cloud_callback(const sensor_msgs::msg::PointCloud2::SharedPtr msg)
 {
   // Be lazy
   if (!find_objects_)
     return;

   // Convert to point cloud
   pcl::PointCloud<pcl::PointXYZRGB>::Ptr cloud =
     boost::make_shared<pcl::PointCloud<pcl::PointXYZRGB>>();
   pcl::fromROSMsg(*msg, *cloud);

   RCLCPP_DEBUG(LOGGER, "Cloud recieved with %d points.",
static_cast<int>(cloud->points.size()));

   // Filter out noisy long-range points
   pcl::PointCloud<pcl::PointXYZRGB>::Ptr cloud_filtered(new pcl::PointCloud<pcl::PointXYZRGB>);
   range_filter_.setInputCloud(cloud);
   range_filter_.filter(*cloud_filtered);
   RCLCPP_DEBUG(LOGGER, "Filtered for range, now %d points.",
             static_cast<int>(cloud_filtered->points.size()));

   // Transform to grounded
   pcl::PointCloud<pcl::PointXYZRGB>::Ptr cloud_transformed(new
pcl::PointCloud<pcl::PointXYZRGB>);
   if (!pcl_ros::transformPointCloud(world_frame_, *cloud_filtered, *cloud_transformed, *buffer_))
   {
     RCLCPP_ERROR(LOGGER, "Error transforming to frame %s", world_frame_.c_str());
     return;
   }

   // Run segmentation
   objects_.clear();
   supports_.clear();
   pcl::PointCloud<pcl::PointXYZRGB> object_cloud;
   pcl::PointCloud<pcl::PointXYZRGB> support_cloud;
   if (debug_)
   {
```

```
    object_cloud.header.frame_id = cloud_transformed->header.frame_id;
    support_cloud.header.frame_id = cloud_transformed->header.frame_id;
  }
  segmentation_->segment(cloud_transformed,
                         objects_,
                         supports_,
                         object_cloud,
                         support_cloud,
                         debug_);

  if (debug_)
  {
    sensor_msgs::msg::PointCloud2 cloud_msg;

    pcl::toROSMsg(object_cloud, cloud_msg);
    object_cloud_pub_->publish(cloud_msg);

    pcl::toROSMsg(support_cloud, cloud_msg);
    support_cloud_pub_->publish(cloud_msg);
  }

  // Ok to continue processing
  find_objects_ = false;
}

rclcpp_action::GoalResponse
handle_goal(const rclcpp_action::GoalUUID & uuid,
  std::shared_ptr<const FindGraspableObjectsAction::Goal> goal_handle)
{
  return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE;
}

rclcpp_action::CancelResponse
handle_cancel(const std::shared_ptr<FindGraspableObjectsGoal> goal_handle)
{
  return rclcpp_action::CancelResponse::ACCEPT;
}
```

This is the `cloud_callback()` function. Within this function, you are doing several things:

- First, transform the incoming PointCloud2 data to PointCloud format for better processing.
- Next, filter the PointCloud data to remove the noisy, long-range points you are not interested in.
- Third, run the **segmentation** process. This is the process that will separate the surface from the detected object/s. The details of this process can be found in the **object_support_segmentation.cpp** file.

- Finally, publish the detections to the related topics for visualization.

In [ ]:

```
void handle_accepted(const std::shared_ptr<FindGraspableObjectsGoal> goal_handle)
 {
   if (!planner_ || !segmentation_)
   {
     // Create planner
     planner_.reset(new ShapeGraspPlanner(this->shared_from_this()));
     // Create perception
     segmentation_.reset(new ObjectSupportSegmentation(this->shared_from_this()));
   }

   // Break off a thread
   std::thread{std::bind(&BasicGraspingPerception::execute, this, _1), goal_handle}.detach();
 }
```

Next, is the `handle_accepted()` function. These lines are also very important. Here, create the **segmentation** object processing the PointCloud data to find any surfaces or objects. The details of this process can be found in the **object_support_segmentation.cpp** file.

You are also creating the **planner** object, generating the grasping poses. However, ignore this part since you have set the **plan_grasps** argument to false.

Finally, break off a thread to process all this data.

In [ ]:

```
void execute(const std::shared_ptr<FindGraspableObjectsGoal> goal_handle)
 {
   auto result = std::make_shared<FindGraspableObjectsAction::Result>();

   // Get objects
   find_objects_ = true;
   rclcpp::Time t = clock_->now();
   while (find_objects_ == true)
   {
     std::this_thread::sleep_for(std::chrono::milliseconds(20));
     if (clock_->now() - t > rclcpp::Duration::from_seconds(3.0))
     {
       find_objects_ = false;
       goal_handle->abort(result);
       RCLCPP_ERROR(LOGGER, "Failed to get camera data in allotted time.");
       return;
     }
   }

   const auto goal = goal_handle->get_goal();
```

```
  // Set object results
  for (size_t i = 0; i < objects_.size(); ++i)
  {
    grasping_msgs::msg::GraspableObject g;
    g.object = objects_[i];
    if (goal->plan_grasps)
    {
      // Plan grasps for object
      planner_->plan(objects_[i], g.grasps);
    }
    result->objects.push_back(g);
  }
  // Set support surfaces
  result->support_surfaces = supports_;

  goal_handle->succeed(result);
}
```

Finally, you have the `execute()` callback function. Each time a new goal is sent to the Action Server, this callback is triggered. So, as you can see, the first thing you do is set the **find_objetcs** variable to **true** so that the cloud callback starts analyzing the PointCloud data. Then, get the detections' results and fill the **result** message with the objects and surfaces obtained from the cloud data analysis.

In [ ]:

```
bool debug_;

std::shared_ptr<tf2_ros::Buffer> buffer_;
std::shared_ptr<tf2_ros::TransformListener> listener_;
std::string world_frame_;

bool find_objects_;
std::vector<grasping_msgs::msg::Object> objects_;
std::vector<grasping_msgs::msg::Object> supports_;

rclcpp::Subscription<sensor_msgs::msg::PointCloud2>::SharedPtr cloud_sub_;
rclcpp::Publisher<sensor_msgs::msg::PointCloud2>::SharedPtr object_cloud_pub_;
rclcpp::Publisher<sensor_msgs::msg::PointCloud2>::SharedPtr support_cloud_pub_;

std::shared_ptr<ShapeGraspPlanner> planner_;
std::shared_ptr<ObjectSupportSegmentation> segmentation_;

rclcpp_action::Server<FindGraspableObjectsAction>::SharedPtr server_;
rclcpp::Clock::SharedPtr clock_;
```

```
pcl::PassThrough<pcl::PointXYZRGB> range_filter_;
```

The final part of the code contains the definitions of all the variables you are using in the script.

5.3 Getting the object position

Great! So you have been able to run your ROS2 Perception nodes to detect objects in the scene, and you've also been able to visualize the objects detected in RVIZ2. But what about their position?

For manipulation, you are interested in knowing the object's position you want to pick. This way, you can plan a motion to send the gripper of your robotic arm to the position of the object to pick. So, how can you get this data?
In this unit, you have already analyzed a little bit of what the `grasping_msgs/action/FindGraspableObjects` message contains. But, first, let's dig a little more. You know that this message returns two arrays:

In [ ]:

```
# Graspable objects found
GraspableObject[] objects

# Additional, non-graspable objects which may be support surfaces
Object[] support_surfaces
```

You are interested in the graspable objects to investigate in this direction. Let's see what this **GraspableObject** message contains:

Execute in Shell

In [ ]:

```
ros2 interface show grasping_msgs/msg/GraspableObject
```

Expected Output

In [ ]:

```
##############################################################
# This message describes an object + grasp data
Object object
moveit_msgs/Grasp[] grasps
```

So this message contains an object's data (in an **Object** message format) plus the grasping data associated with this object. We have already stated that you will ignore the grasping in this unit, so let's focus on the Object data.

Execute in Shell

In [ ]:

```
ros2 interface show grasping_msgs/msg/Object
```

Expected Output

In [ ]:

```
##############################################################
# This message describes an object.

# Many of the geometric items below lack a stamp/frame_id,
# header stamp/frame_id should be used there
std_msgs/Header header

# An object might have a name
string name

# An object might have a known (named) support surface
string support_surface

# Objects might have properties, such as type/class, color, etc.
ObjectProperty[] properties

##############################################################
# Objects have many possible descriptions
#  The following are the possible description formats

# Perception modules often represent an object as a cluster of points
#  Is considered valid if the number of points > 0
sensor_msgs/PointCloud2 point_cluster

# MoveIt prefers solid primitives or meshes as a description of objects
shape_msgs/SolidPrimitive[] primitives
geometry_msgs/Pose[] primitive_poses

shape_msgs/Mesh[] meshes
geometry_msgs/Pose[] mesh_poses

# An object representing a support surface might be described by a plane
# Is considered valid if coefficients are not all 0s.
shape_msgs/Plane surface
```

And voilà! You've found the treasure!

You can see that there is a lot of data in this Object message. However, among all these variables, focus on one in particular:

In [ ]:

```
geometry_msgs/Pose[] primitive_poses
```

The **primitive_poses** variable contains the data of the poses of the object in the format of a **geometry_msgs/Pose** message. In manipulation, you know as **primitive** any primitive shapes, for

instance, boxes, cylinders, cones, spheres, and planes. In general, MoveIt prefers solid primitives or meshes to describe.

Execute in Shell

In [ ]:

```
ros2 interface show geometry_msgs/msg/Pose
```

Expected Output

In [ ]:

```
# A representation of pose in free space, composed of position and orientation.

Point position
Quaternion orientation
```

Finally, you can get to the data of the object's position.

- Exercise 5.2 -

Create a new ROS2 package named **get_pose_client**. Inside this package, create a new C++ script, named **get_pose_client.cpp**, that does the following:

- It creates an Action Client that connects to the **/find_objects** Action Server.
- It sends a goal to the Action Server to get the detected object's data. Set the **plan_grasps** variable to **false**.
- It receives the result from the Action Server and processes it just to print the [X, Y] position of the object of interest.

- End of Exercise 5.2 -

- Expected Result for Exercise 5.2 -

```
user:~/ros2_ws$ ros2 run get_pose_client get_pose_client
[INFO] [1635153416.076482633] [minimal_action_client]: Sending goal
[INFO] [1635153416.077067221] [minimal_action_client]: Goal accepted by server, waiting for result
[INFO] [1635153416.710607953] [minimal_action_client]: Result received
[INFO] [1635153416.710731155] [minimal_action_client]: X: 0.488025
[INFO] [1635153416.710769263] [minimal_action_client]: Y: 0.149253
```

- End of Expected Result -

- Solution for Exercise 5.2 -

Execute in Shell #1

In [ ]:

```
ros2 pkg create get_pose_client --dependencies grasping_msgs rclcpp rclcpp_action
```

CMakeLists.txt

In [ ]:

```
cmake_minimum_required(VERSION 3.8)
project(get_pose_client)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()
```

```
# find dependencies
find_package(ament_cmake REQUIRED)
find_package(grasping_msgs REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_action REQUIRED)

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # comment the line when a copyright and license is added to all source files
  set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # comment the line when this package is in a git repo and when
  # a copyright and license is added to all source files
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

add_executable(get_pose_client src/get_pose_client.cpp)
ament_target_dependencies(get_pose_client
  "rclcpp"
  "rclcpp_action"
  "grasping_msgs")

install(TARGETS
  get_pose_client
  DESTINATION lib/${PROJECT_NAME})

ament_package()
```

get_pose_client.cpp

In [ ]:

```
#include "grasping_msgs/action/find_graspable_objects.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
#include <inttypes.h>
#include <iostream>
#include <memory>
#include <string>

class GetPoseClient : public rclcpp::Node {
public:
  using Find = grasping_msgs::action::FindGraspableObjects;
  using GoalHandleFind = rclcpp_action::ClientGoalHandle<Find>;

  explicit GetPoseClient(
```

```
    const rclcpp::NodeOptions &node_options = rclcpp::NodeOptions())
    : Node("get_pose_client", node_options), goal_done_(false) {
  this->client_ptr_ = rclcpp_action::create_client<Find>(
      this->get_node_base_interface(), this->get_node_graph_interface(),
      this->get_node_logging_interface(),
      this->get_node_waitables_interface(), "find_objects");

  this->timer_ =
      this->create_wall_timer(std::chrono::milliseconds(500),
                              std::bind(&GetPoseClient::send_goal, this));
}

bool is_goal_done() const { return this->goal_done_; }

void send_goal() {
  using namespace std::placeholders;

  this->timer_->cancel();

  this->goal_done_ = false;

  if (!this->client_ptr_) {
    RCLCPP_ERROR(this->get_logger(), "Action client not initialized");
  }

  if (!this->client_ptr_->wait_for_action_server(std::chrono::seconds(10))) {
    RCLCPP_ERROR(this->get_logger(),
                 "Action server not available after waiting");
    this->goal_done_ = true;
    return;
  }

  auto goal_msg = Find::Goal();
  goal_msg.plan_grasps = false;

  RCLCPP_INFO(this->get_logger(), "Sending goal");

  auto send_goal_options = rclcpp_action::Client<Find>::SendGoalOptions();
  send_goal_options.goal_response_callback =
      std::bind(&GetPoseClient::goal_response_callback, this, _1);
  send_goal_options.feedback_callback =
      std::bind(&GetPoseClient::feedback_callback, this, _1, _2);
  send_goal_options.result_callback =
      std::bind(&GetPoseClient::result_callback, this, _1);
  auto goal_handle_future =
      this->client_ptr_->async_send_goal(goal_msg, send_goal_options);
```

```
  }

private:
  rclcpp_action::Client<Find>::SharedPtr client_ptr_;
  rclcpp::TimerBase::SharedPtr timer_;
  bool goal_done_;

  void goal_response_callback(const GoalHandleFind::SharedPtr &goal_handle) {
    if (!goal_handle) {
      RCLCPP_ERROR(this->get_logger(), "Goal was rejected by server");
    } else {
      RCLCPP_INFO(this->get_logger(),
                  "Goal accepted by server, waiting for result");
    }
  }

  void feedback_callback(GoalHandleFind::SharedPtr,
                         const std::shared_ptr<const Find::Feedback> feedback) {
    RCLCPP_INFO(this->get_logger(), "Ignoring feedback...");
  }

  void result_callback(const GoalHandleFind::WrappedResult &result) {
    this->goal_done_ = true;
    switch (result.code) {
    case rclcpp_action::ResultCode::SUCCEEDED:
      break;
    case rclcpp_action::ResultCode::ABORTED:
      RCLCPP_ERROR(this->get_logger(), "Goal was aborted");
      return;
    case rclcpp_action::ResultCode::CANCELED:
      RCLCPP_ERROR(this->get_logger(), "Goal was canceled");
      return;
    default:
      RCLCPP_ERROR(this->get_logger(), "Unknown result code");
      return;
    }

    RCLCPP_INFO(this->get_logger(), "Result received");
    RCLCPP_INFO(this->get_logger(), "X: %f",
                result.result->objects[0].object.primitive_poses[0].position.x);
    RCLCPP_INFO(this->get_logger(), "Y: %f",
                result.result->objects[0].object.primitive_poses[0].position.y);
    //}
  }
}; // class GetPoseClient
```

```
int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  auto action_client = std::make_shared<GetPoseClient>();

  while (!action_client->is_goal_done()) {
    rclcpp::spin_some(action_client);
  }

  rclcpp::shutdown();
  return 0;
}
```

- End of Solution for Exercise 5.2 -

- Exercise 5.3 -

In the previous exercise, you are directly getting the 1st element of the **object's** array:

In [ ]:

```
result.result->objects[0]
```

But what if there are more than 1 object detected? How can you differentiate between them?
Let's spawn into the world a 2nd object. Create a new file inside `~/ros2_ws/src` and paste the below code:

grasp_box2.urdf

In [ ]:

```
<robot name="grasp_box">

    <!-- Colours for RVIZ for geometric elements -->
    <material name="blue">
        <color rgba="0 0 0.8 1"/>
    </material>
    <material name="red">
        <color rgba="0.8 0 0 1"/>
    </material>
    <material name="green">
        <color rgba="0 0.8 0 1"/>
    </material>
    <material name="grey">
        <color rgba="0.75 0.75 0.75 1"/>
    </material>
    <material name="white">
        <color rgba="1.0 1.0 1.0 1"/>
    </material>
    <material name="black">
        <color rgba="0 0 0 1"/>
    </material>
```

```
        <!-- * * * Link Definitions * * * -->

    <link name="grasp_box2_base_link">
            <inertial>
        <origin xyz="0 0 0" rpy="0 0 0"/>
        <mass value="0.5" />
        <inertia ixx="0.000133333333333" ixy="0.0" ixz="0.0" iyy="0.000133333333333" iyz="0.0"
izz="0.000133333333333"/>
      </inertial>
      <collision>
        <origin xyz="0 0 0" rpy="0 0 0"/>
        <geometry>
          <box size="0.1 0.1 0.1"/>
        </geometry>
      </collision>
      <visual>
        <origin rpy="0.0 0 0" xyz="0 0 0"/>
        <geometry>
          <box size="0.1 0.1 0.1"/>
        </geometry>
        <material name="blue"/>
      </visual>
    </link>

    <gazebo reference="grasp_box2_base_link">
      <kp>100000.0</kp>
      <kd>100.0</kd>
      <mu1>0.25</mu1>
      <mu2>0.25</mu2>
      <material>Gazebo/Blue</material>
    </gazebo>

</robot>
```

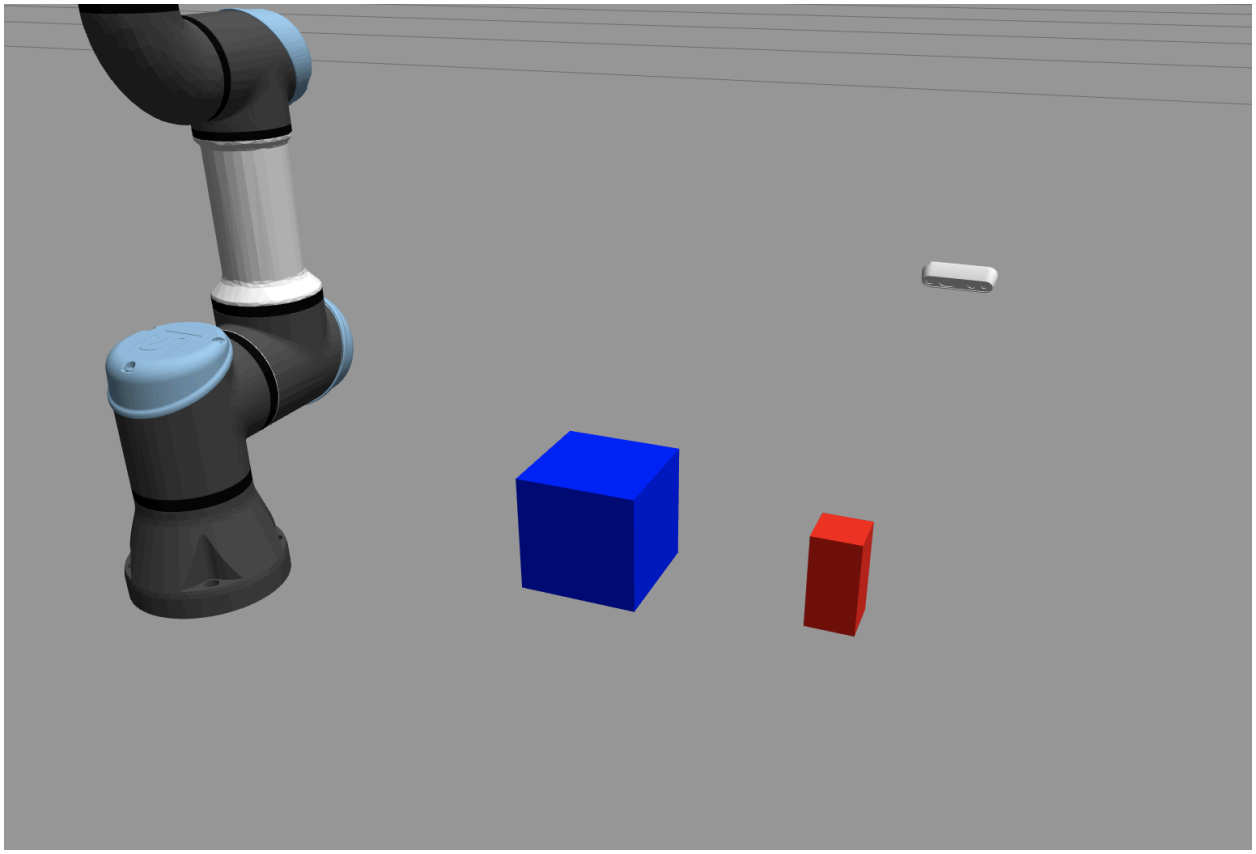
Spawn it into the world with the following command:

Execute in Shell

In [ ]:

```
ros2 run gazebo_ros spawn_entity.py -file ~/ros2_ws/src/grasp_box2.urdf -x 0.3 -y 0.13 -z 0.1 -entity
grasp_box2
```

You should end up with something like this:



Now, update the code in **get_pose_client.cpp** so that it now does the following:

- When you receive the **object's** array, you analyze the **dimensions** of each object in the array. Then, you will only get the object's position [X, Y], whose dimensions match the object's dimensions of interest.

- Notes -

You can access the dimensions data of the object in the **primitives** variable of the **Object** message.

In [ ]:

shape_msgs/SolidPrimitive[] primitives

Execute in Shell #4

In [ ]:

ros2 interface show shape_msgs/msg/SolidPrimitive

Expected Output

In [ ]:

```
# Defines box, sphere, cylinder, and cone.
# All shapes are defined to have their bounding boxes centered around 0,0,0.

uint8 BOX=1
uint8 SPHERE=2
uint8 CYLINDER=3
```

```
uint8 CONE=4

# The type of the shape
uint8 type

# The dimensions of the shape
float64[<=3] dimensions  # At no point will dimensions have a length > 3.

# The meaning of the shape dimensions: each constant defines the index in the 'dimensions' array.

# For type BOX, the X, Y, and Z dimensions are the length of the corresponding sides of the box.
uint8 BOX_X=0
uint8 BOX_Y=1
uint8 BOX_Z=2

# For the SPHERE type, only one component is used, giving the sphere's radius.
uint8 SPHERE_RADIUS=0

# The center line is oriented along the Z-axis for the CYLINDER and CONE types,
# Therefore, the CYLINDER_HEIGHT (CONE_HEIGHT) component of dimensions gives the
# height of the cylinder (cone).
# The CYLINDER_RADIUS (CONE_RADIUS) component of dimensions provides the radius of
# the base of the cylinder (cone).
# Cone and cylinder primitives are defined to be circular. Therefore, the tip of the cone
# is pointing up along the +Z axis.

uint8 CYLINDER_HEIGHT=0
uint8 CYLINDER_RADIUS=1

uint8 CONE_HEIGHT=0
uint8 CONE_RADIUS=1
```

- End of Notes -

- End of Exercise 5.3 -

- Expected Result for Exercise 5.3 -

```
user:~/ros2_ws$ ros2 run get_pose_client get_pose_client
[INFO] [1635153416.076482633] [minimal_action_client]: Sending goal
[INFO] [1635153416.077067221] [minimal_action_client]: Goal accepted by server, waiting for result
[INFO] [1635153416.710607953] [minimal_action_client]: Result received
[INFO] [1635153416.710731155] [minimal_action_client]: X: 0.488025
[INFO] [1635153416.710769263] [minimal_action_client]: Y: 0.149253
```

- End of Expected Result -

- Solution for Exercise 5.3 -

UPDATED get_pose_client.cpp v1

In [ ]:

```
#include "grasping_msgs/action/find_graspable_objects.hpp"
#include "rclcpp/rclcpp.hpp"
```

```cpp
#include "rclcpp_action/rclcpp_action.hpp"
#include <inttypes.h>
#include <iostream>
#include <memory>
#include <string>

class GetPoseClient : public rclcpp::Node {
public:
  using Find = grasping_msgs::action::FindGraspableObjects;
  using GoalHandleFind = rclcpp_action::ClientGoalHandle<Find>;

  explicit GetPoseClient(
      const rclcpp::NodeOptions &node_options = rclcpp::NodeOptions())
      : Node("get_pose_client", node_options), goal_done_(false) {
    this->client_ptr_ = rclcpp_action::create_client<Find>(
        this->get_node_base_interface(), this->get_node_graph_interface(),
        this->get_node_logging_interface(),
        this->get_node_waitables_interface(), "find_objects");

    this->timer_ =
        this->create_wall_timer(std::chrono::milliseconds(500),
                                std::bind(&GetPoseClient::send_goal, this));
  }

  bool is_goal_done() const { return this->goal_done_; }

  void send_goal() {
    using namespace std::placeholders;

    this->timer_->cancel();

    this->goal_done_ = false;

    if (!this->client_ptr_) {
      RCLCPP_ERROR(this->get_logger(), "Action client not initialized");
    }

    if (!this->client_ptr_->wait_for_action_server(std::chrono::seconds(10))) {
      RCLCPP_ERROR(this->get_logger(),
                   "Action server not available after waiting");
      this->goal_done_ = true;
      return;
    }

    auto goal_msg = Find::Goal();
    goal_msg.plan_grasps = false;
```

```
    RCLCPP_INFO(this->get_logger(), "Sending goal");

    auto send_goal_options = rclcpp_action::Client<Find>::SendGoalOptions();
    send_goal_options.goal_response_callback =
        std::bind(&GetPoseClient::goal_response_callback, this, _1);
    send_goal_options.feedback_callback =
        std::bind(&GetPoseClient::feedback_callback, this, _1, _2);
    send_goal_options.result_callback =
        std::bind(&GetPoseClient::result_callback, this, _1);
    auto goal_handle_future =
        this->client_ptr_->async_send_goal(goal_msg, send_goal_options);
  }

private:
  rclcpp_action::Client<Find>::SharedPtr client_ptr_;
  rclcpp::TimerBase::SharedPtr timer_;
  bool goal_done_;

  void goal_response_callback(const GoalHandleFind::SharedPtr &goal_handle) {
    if (!goal_handle) {
      RCLCPP_ERROR(this->get_logger(), "Goal was rejected by server");
    } else {
      RCLCPP_INFO(this->get_logger(),
                  "Goal accepted by server, waiting for result");
    }
  }

  void feedback_callback(GoalHandleFind::SharedPtr,
                         const std::shared_ptr<const Find::Feedback> feedback) {
    RCLCPP_INFO(this->get_logger(), "Ignoring feedback...");
  }

  void result_callback(const GoalHandleFind::WrappedResult &result) {
    this->goal_done_ = true;
    switch (result.code) {
    case rclcpp_action::ResultCode::SUCCEEDED:
      break;
    case rclcpp_action::ResultCode::ABORTED:
      RCLCPP_ERROR(this->get_logger(), "Goal was aborted");
      return;
    case rclcpp_action::ResultCode::CANCELED:
      RCLCPP_ERROR(this->get_logger(), "Goal was canceled");
      return;
    default:
      RCLCPP_ERROR(this->get_logger(), "Unknown result code");
```

```
      return;
    }

    RCLCPP_INFO(this->get_logger(), "Result received");
    for (auto object : result.result->objects) {
      RCLCPP_INFO(this->get_logger(), "Type: %d",
                  object.object.primitives[0].type);
      RCLCPP_INFO(this->get_logger(), "x: %f",
                  object.object.primitives[0].dimensions[0]);
      RCLCPP_INFO(this->get_logger(), "Y: %f",
                  object.object.primitives[0].dimensions[1]);
      RCLCPP_INFO(this->get_logger(), "Z: %f",
                  object.object.primitives[0].dimensions[2]);
    }
    //}
  }
}; // class GetPoseClient

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  auto action_client = std::make_shared<GetPoseClient>();

  while (!action_client->is_goal_done()) {
    rclcpp::spin_some(action_client);
  }

  rclcpp::shutdown();
  return 0;
}
```

With the above code, you will get an output similar to this:

```
user:~/ros2_ws$ ros2 run get_pose_client get_pose_client
[INFO] [1695659024.541615153] [get_pose_client]: Sending goal
[INFO] [1695659024.542715606] [get_pose_client]: Goal accepted by server, waiting for result
[INFO] [1695659025.976735648] [get_pose_client]: Result received
[INFO] [1695659025.976827319] [get_pose_client]: Type: 1
[INFO] [1695659025.976863062] [get_pose_client]: x: 0.098531
[INFO] [1695659025.976899461] [get_pose_client]: Y: 0.099293
[INFO] [1695659025.976956924] [get_pose_client]: Z: 0.046119
[INFO] [1695659025.977005879] [get_pose_client]: Type: 1
[INFO] [1695659025.977039494] [get_pose_client]: x: 0.038075
[INFO] [1695659025.977071370] [get_pose_client]: Y: 0.000003
[INFO] [1695659025.977103517] [get_pose_client]: Z: 0.042329
```

As you can see, you are printing some data related to the objects detected. The data you are printing is:

- Type: It represents the primitive shape of the object.

In [ ]:

BOX=1

SPHERE=2
CYLINDER=3
CONE=4

- The [X, Y, Z] dimensions of the object.

With this knowledge, you can now figure out how to filter the data to get only the desired object. One possible filter is the **type** of object. For example, you know that your object is box-shaped (type 1), so you can already filter out any object which doesn't have this shape.

Also, you know the dimensions of the object from the URDF file:

In [ ]:

```
<box size="0.04 0.04 0.08"/>
```

Therefore, anything which is bigger than this can also be discarded. For instance, in this example, one of the objects detected is way bigger than this (0.19x0.12x0.29).

With all this data, create the final version of your code:

UPDATED get_pose_client.cpp v2

In [ ]:

```
#include "grasping_msgs/action/find_graspable_objects.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
#include <inttypes.h>
#include <iostream>
#include <memory>
#include <string>

class GetPoseClient : public rclcpp::Node {
public:
  using Find = grasping_msgs::action::FindGraspableObjects;
  using GoalHandleFind = rclcpp_action::ClientGoalHandle<Find>;

  explicit GetPoseClient(
      const rclcpp::NodeOptions &node_options = rclcpp::NodeOptions())
      : Node("get_pose_client", node_options), goal_done_(false) {
    this->client_ptr_ = rclcpp_action::create_client<Find>(
        this->get_node_base_interface(), this->get_node_graph_interface(),
        this->get_node_logging_interface(),
        this->get_node_waitables_interface(), "find_objects");

    this->timer_ =
        this->create_wall_timer(std::chrono::milliseconds(500),
                                std::bind(&GetPoseClient::send_goal, this));
  }
```

```
  bool is_goal_done() const { return this->goal_done_; }

  void send_goal() {
    using namespace std::placeholders;

    this->timer_->cancel();

    this->goal_done_ = false;

    if (!this->client_ptr_) {
      RCLCPP_ERROR(this->get_logger(), "Action client not initialized");
    }

    if (!this->client_ptr_->wait_for_action_server(std::chrono::seconds(10))) {
      RCLCPP_ERROR(this->get_logger(),
                   "Action server not available after waiting");
      this->goal_done_ = true;
      return;
    }

    auto goal_msg = Find::Goal();
    goal_msg.plan_grasps = false;

    RCLCPP_INFO(this->get_logger(), "Sending goal");

    auto send_goal_options = rclcpp_action::Client<Find>::SendGoalOptions();
    send_goal_options.goal_response_callback =
        std::bind(&GetPoseClient::goal_response_callback, this, _1);
    send_goal_options.feedback_callback =
        std::bind(&GetPoseClient::feedback_callback, this, _1, _2);
    send_goal_options.result_callback =
        std::bind(&GetPoseClient::result_callback, this, _1);
    auto goal_handle_future =
        this->client_ptr_->async_send_goal(goal_msg, send_goal_options);
  }

private:
  rclcpp_action::Client<Find>::SharedPtr client_ptr_;
  rclcpp::TimerBase::SharedPtr timer_;
  bool goal_done_;

  void goal_response_callback(const GoalHandleFind::SharedPtr &goal_handle) {
    if (!goal_handle) {
      RCLCPP_ERROR(this->get_logger(), "Goal was rejected by server");
    } else {
      RCLCPP_INFO(this->get_logger(),
```

```
                "Goal accepted by server, waiting for result");
    }
  }

  void feedback_callback(GoalHandleFind::SharedPtr,
                         const std::shared_ptr<const Find::Feedback> feedback) {
    RCLCPP_INFO(this->get_logger(), "Ignoring feedback...");
  }

  void result_callback(const GoalHandleFind::WrappedResult &result) {
    this->goal_done_ = true;
    switch (result.code) {
    case rclcpp_action::ResultCode::SUCCEEDED:
      break;
    case rclcpp_action::ResultCode::ABORTED:
      RCLCPP_ERROR(this->get_logger(), "Goal was aborted");
      return;
    case rclcpp_action::ResultCode::CANCELED:
      RCLCPP_ERROR(this->get_logger(), "Goal was canceled");
      return;
    default:
      RCLCPP_ERROR(this->get_logger(), "Unknown result code");
      return;
    }

    RCLCPP_INFO(this->get_logger(), "Result received");
    for (auto object : result.result->objects) {
      if (object.object.primitives[0].type == 1 &&
          object.object.primitives[0].dimensions[0] < 0.05 &&
          object.object.primitives[0].dimensions[1] < 0.05 &&
          object.object.primitives[0].dimensions[2] < 0.1) {
        RCLCPP_INFO(this->get_logger(), "X: %f",
                    object.object.primitive_poses[0].position.x);
        RCLCPP_INFO(this->get_logger(), "Y: %f",
                    object.object.primitive_poses[0].position.y);
      }
    }
    //}
  }
}; // class GetPoseClient

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  auto action_client = std::make_shared<GetPoseClient>();

  while (!action_client->is_goal_done()) {
```

```
    rclcpp::spin_some(action_client);
  }

  rclcpp::shutdown();
  return 0;
}
```

- End of Solution -

5.4 Pick & Place with Perception

In the previous Pick & Place example, you sent the robot to a hardcoded pose in order to pick the object. But in real life scenarios, it is very likely that you won't know the pose of the objects to grasp. Therefore, in the next exercise you will have to update your Pick&Place application in order to use perception to detect the pose of the object.

- Exercise 5.4 -

Update the `pick_place.cpp` script you created in the previous unit. Instead of hardcoding the pose of the object, use the Perception action in order to get its pose.

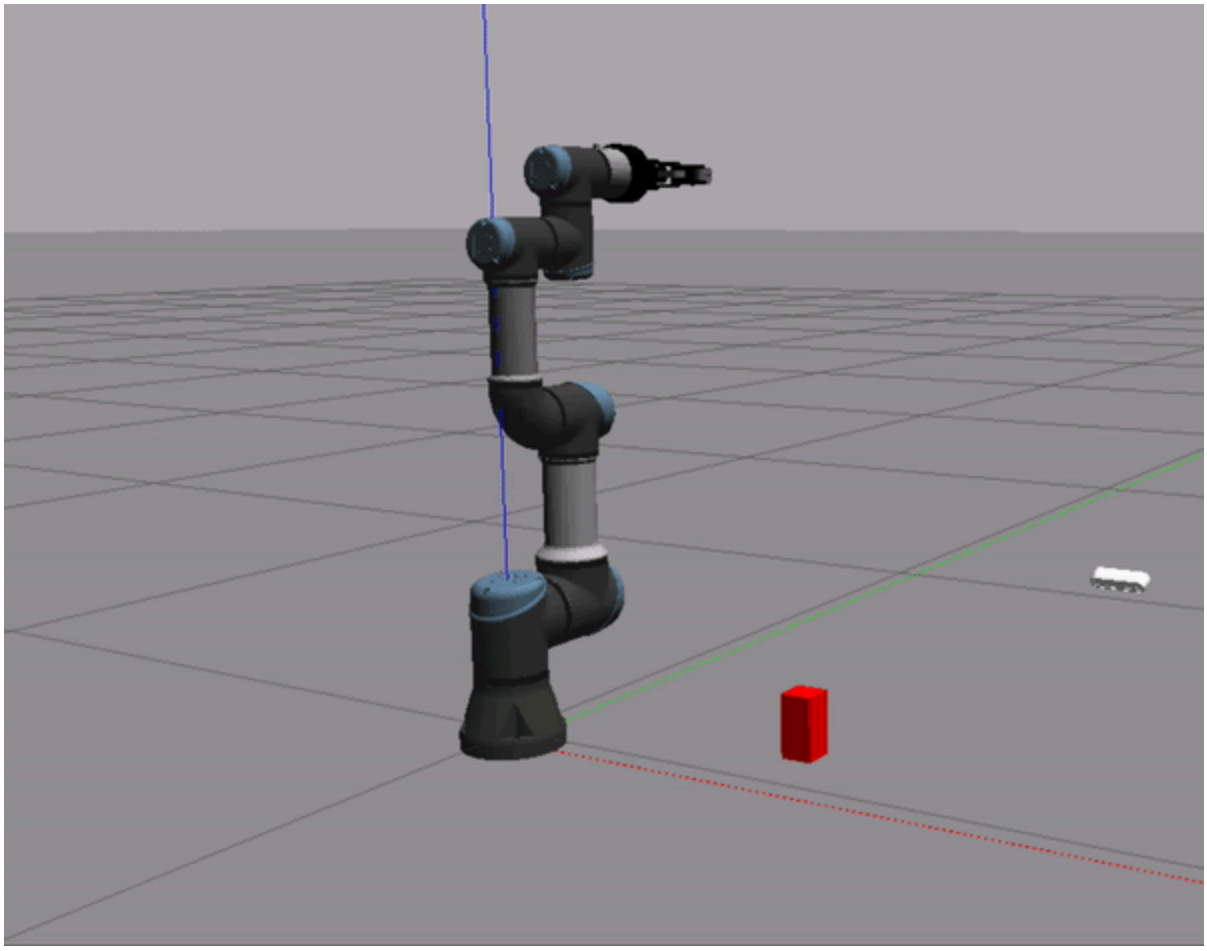
Follow the next steps in order to complete the exercise:

1.- Add an action client that sends a goal to the `/find_objects` action server in order to search for objects in the environment.

2.- Get the Pose of the object to pick and use it to send the end-effector near the object (right before the approach motion) .

- End of Exercise 5.4 -

- Expected Result for Exercise 5.4 -



- End of Expected Result -

English proofread